Before/after and threshold designs

Lecture notes

Foundations of Regression and Predictive Modeling

Table of contents

When you cannot randomize . . . 1
Case 1: the courier pay guarantee . . . 1
Why before/after is not enough . . . 2
The difference-in-differences logic . . . 3
Difference-in-differences as regression . . . 4
Parallel trends . . . 5
Case 2: customer success managers at a threshold . . . 6
The threshold as a local experiment . . . 7
The RD model . . . 7
RD assumptions and the bunching check . . . 9
Summary . . . 9

When you cannot randomize

The last two modules assumed you got to design the experiment: flip coins, form blocks, assign treatments. Often you cannot. The launch already happened, or randomizing would be unfair, impractical, or forbidden. This module covers the two most useful quasi-experimental designs — situations where the world's own structure supplies a credible comparison even though nobody randomized anything.

Difference-in-differences uses before/after data plus a comparison group. **Regression discontinuity** uses a threshold rule plus the observations near the cutoff. In both designs the estimate is a single regression coefficient, and the regression itself is easy. The real work is the argument that the comparison encoded in the model is a fair one. That argument rests on assumptions, and the assumptions — not the p-values — are where these analyses succeed or fail.

Case 1: the courier pay guarantee

A restaurant delivery marketplace is losing couriers in San Antonio, so operations proposes a guaranteed hourly minimum: whenever fares fall short of $18 per hour while a courier is active, the company tops up the difference. The guarantee is expensive, and the CFO wants evidence before any statewide rollout. In mid-June it launches in San Antonio only.

The data are weekly counts of active couriers for San Antonio and for Fort Worth, a comparable market that did not get the guarantee — 40 weeks in all, 24 before launch and 16 after:

```
couriers <- read_csv(file.path(module_dir, "courier_pay.csv"))
couriers |> head(4)
```

```
# A tibble: 4 x 6
  city         week week_start treated  post active_couriers
  <chr>       <dbl> <date>       <dbl> <dbl>           <dbl>
1 fort_worth      1 2024-01-01       0     0            1596
2 san_antonio     1 2024-01-01       1     0            1338
3 fort_worth      2 2024-01-08       0     0            1614
4 san_antonio     2 2024-01-08       1     0            1359
```

`treated` marks the guarantee city; `post` marks weeks after the mid-June launch.

Why before/after is not enough

The tempting analysis compares San Antonio to itself:

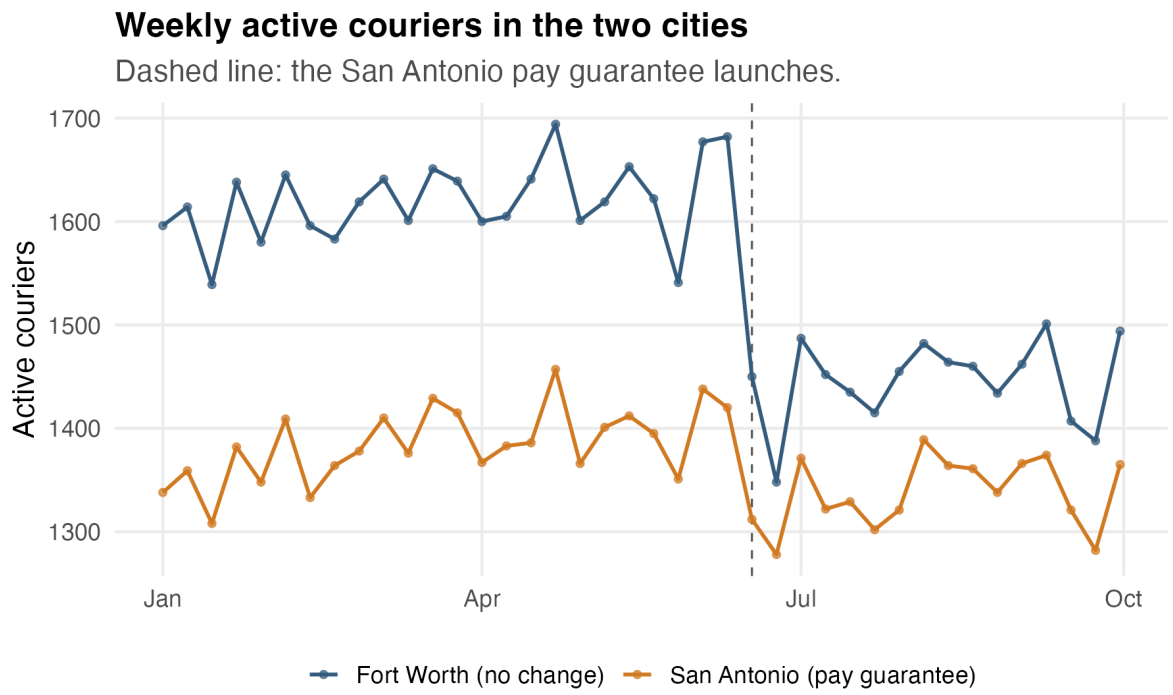
```
couriers |>
  filter(city == "san_antonio") |>
  group_by(post) |>
  summarize(mean_couriers = mean(active_couriers))
```

```
# A tibble: 2 x 2
   post mean_couriers
  <dbl>         <dbl>
1     0         1384.
2     1         1337.
```

San Antonio *lost* about 47 couriers per week after launch. Read naively, the guarantee failed.

The before/after comparison treats time as the only thing that changed at launch. But in mid-June a rideshare company began a statewide driver-recruitment promotion, and delivery couriers everywhere started defecting. The guarantee launched into a headwind, and a before/after comparison cannot tell the policy apart from the headwind.

Fort Worth breaks the tie, because Fort Worth got the headwind but not the guarantee:



The difference-in-differences logic

Four means carry the whole design:

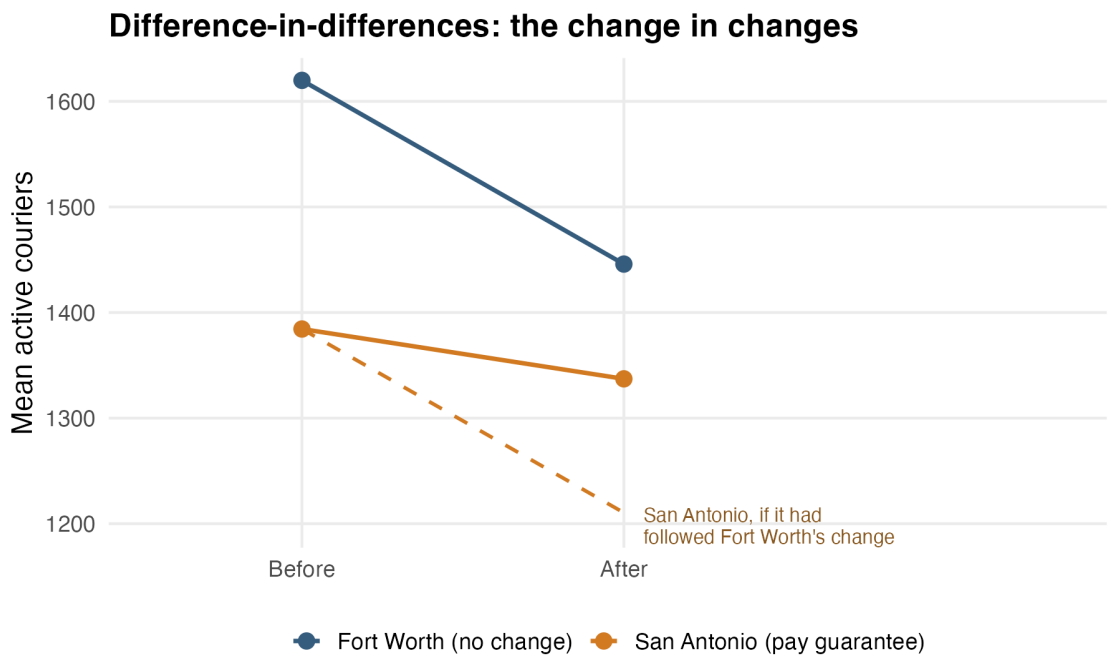
```
couriers |>
  group_by(city, post) |>
  summarize(mean_couriers = mean(active_couriers), .groups = "drop")
```

```
# A tibble: 4 x 3
  city         post mean_couriers
  <chr>       <dbl>         <dbl>
1 fort_worth      0         1620.
2 fort_worth      1         1446.
3 san_antonio     0         1384.
4 san_antonio     1         1337.
```

Fort Worth fell from about 1620 to 1446 — a change of −174, which estimates the headwind. San Antonio fell from about 1384 to 1337 — a change of −47, which mixes the headwind with the guarantee. The difference in the differences isolates the guarantee:

$$(-47) - (-174) = +127 \text{ couriers per week.}$$

The counterfactual claim is visible in the picture: had San Antonio followed Fort Worth's slide, it would have landed near 1210. It landed at 1337 instead.



Difference-in-differences as regression

The regression version is a group-by-post interaction:

```
did_model <- lm(active_couriers ~ treated * post, data = couriers)
tidy(did_model, conf.int = TRUE)
```

```
# A tibble: 4 x 7
  term         estimate std.error statistic   p.value conf.low conf.high
  <chr>           <dbl>     <dbl>     <dbl>     <dbl>    <dbl>     <dbl>
1 (Intercept)     1620.      7.66    211.   4.81e-107   1605.      1635.
2 treated         -235.     10.8     -21.7  2.34e- 34   -257.      -214.
3 post            -174.     12.1     -14.4  2.31e- 23   -198.      -150.
4 treated:post     127.     17.1       7.40 1.51e- 10     92.7      161.
```

Four coefficients, four cell means:

- the intercept is the baseline cell, Fort Worth before launch;
- `treated` is San Antonio's permanent level gap — it was always the smaller market, and DiD does not care;
- `post` is the headwind — what happened to the untreated city;
- `treated:post` is the change in changes, about +127 couriers with a 95% interval of roughly [93, 161].

The first three coefficients are scenery. The interaction is the answer.

The market-wide weekly wiggles are shared shocks — nuisance variation with a name — and the fixed-effects trick from last module absorbs them:

```r
did_week_fe <- lm(
  active_couriers ~ treated + factor(week) + treated:post,
  data = couriers
)
tidy(did_week_fe, conf.int = TRUE) |> filter(!str_detect(term, "week"))
```

```
# A tibble: 3 x 7
  term         estimate std.error statistic  p.value conf.low conf.high
  <chr>           <dbl>     <dbl>     <dbl>    <dbl>    <dbl>     <dbl>
1 (Intercept)     1585.      9.12     174.  1.00e-56    1566.     1603.
2 treated         -235.      3.65     -64.5 1.91e-40    -243.     -228.
3 treated:post     127.      5.77      22.0 3.31e-23     115.      138.
```

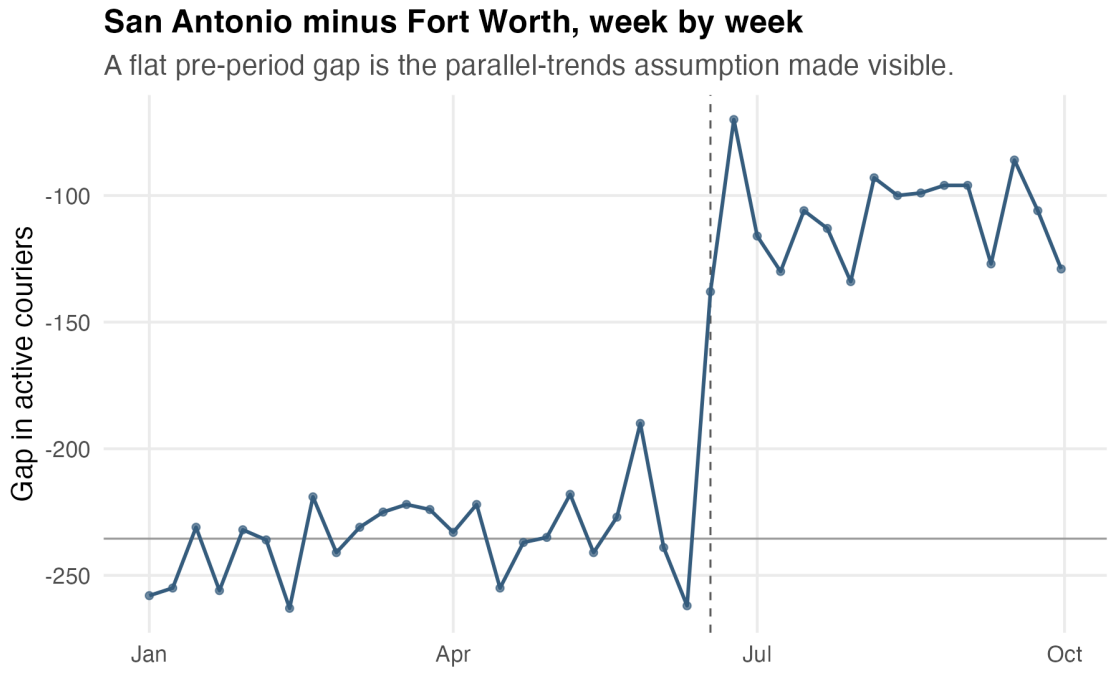
Same estimate, tighter interval — about $[115, 138]$. Blocks, fixed effects, and DiD are one toolkit, not three.

Parallel trends

The assumption purchased with this design, in ordinary language:

> if the guarantee had never launched, San Antonio's courier count would have changed the way Fort Worth's did.

This is a claim about counterfactual *changes*, not about levels; the cities can differ permanently. It is unverifiable in the post period — that is exactly the counterfactual — but it leaves fingerprints in the pre period. If the cities genuinely move in parallel absent treatment, the week-by-week gap between them should be flat before launch:



Six months of flat pre-period gap does not prove parallel trends, but it makes them plausible, and a drifting pre-period gap would have killed the analysis before it started.

The threats worth checking do not appear in the regression output:

- **A local co-event.** If San Antonio also raised courier parking fines at launch time, the interaction absorbs that too, wrongly.
- **Composition change.** If the guarantee attracts new couriers while old ones keep quitting, "active couriers" blends retention with recruitment.
- **Anticipation.** If couriers heard rumors in May and stuck around, the "before" period is contaminated.
- **Targeted timing.** If San Antonio got the policy *because* it was suddenly bleeding couriers, mean reversion can masquerade as a treatment effect.
- **Two markets.** One treated and one control city is the minimum viable design. The interval leans entirely on these two markets moving together for forty weeks; every added control market — and every added treated market in a staggered rollout — makes the uncertainty statement more credible.

These are questions about how the launch was decided, answered by asking people, not software.

Case 2: customer success managers at a threshold

An Austin B2B software company assigns a named customer success manager (CSM) to every account whose prior-year ARR is $20,000 or more; smaller accounts get self-serve support. CSMs are expensive, and the CFO wants to know whether the program improves retention. The outcome is net revenue retention (NRR): next-year spend as a percent of prior-year spend.

```
accounts <- read_csv(file.path(module_dir, "csm_accounts.csv"))
accounts |> head(4)
```

```
# A tibble: 4 x 4
  account_id arr_prior_k   csm renewal_spend_pct
  <chr>            <dbl> <dbl>             <dbl>
1 K8318             25.8     1             105.
2 K6355             16.4     0              96.3
3 K3812             18.8     0              95.6
4 K3865             26.1     1             100
```

The tempting analysis compares all CSM accounts against all non-CSM accounts:

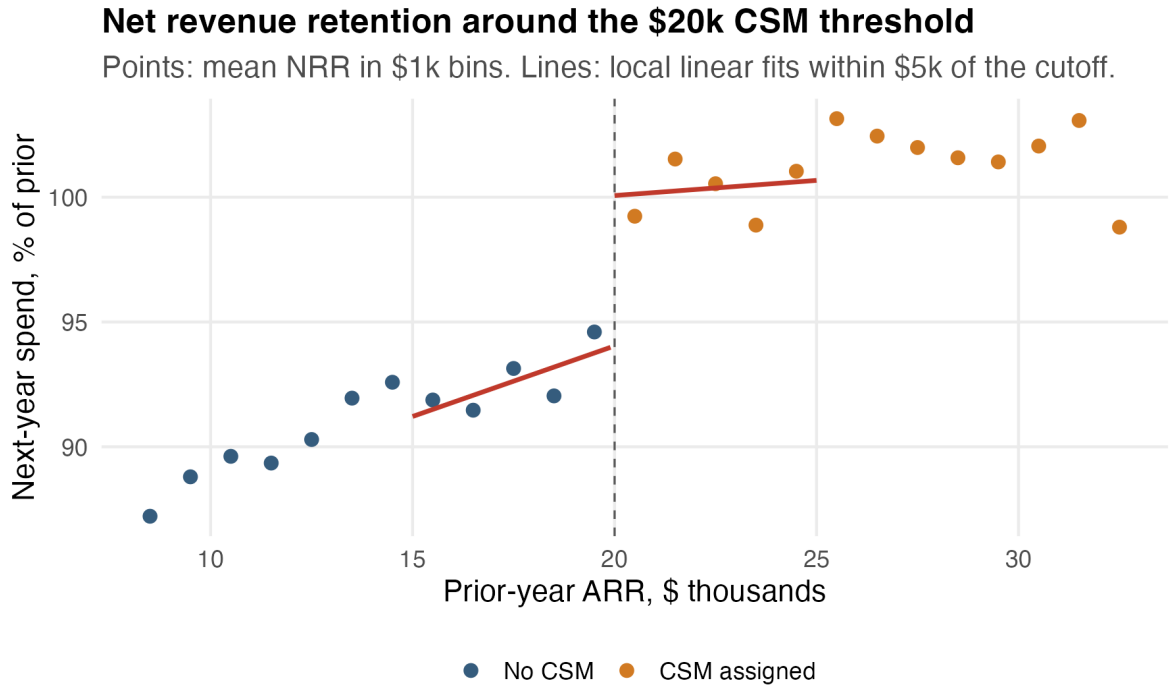
```
accounts |>
  group_by(csm) |>
  summarize(n = n(), mean_nrr = mean(renewal_spend_pct))
```

```
# A tibble: 2 x 3
    csm     n mean_nrr
  <dbl> <int>    <dbl>
1     0   610     91.1
2     1   590    101.
```

A gap of about +10.3 points. But CSM accounts are, by the rule itself, the bigger accounts, and bigger accounts retain better with or without help. The comparison is confounded by the very variable that assigns the treatment.

The threshold as a local experiment

Consider an account at $19.8k and one at $20.2k. As businesses they are interchangeable — a few hundred dollars of last year's contracts separates them — yet the rule treats them completely differently. Right at the cutoff, the rule behaves like a coin flip. That is the regression discontinuity idea: the threshold runs a local experiment for free.



Two things are true at once in this picture: retention rises smoothly with account size, and it jumps at the line where CSMs begin. The smooth part is size; the jump is the program. Keeping them separate is the entire craft of RD.

The RD model

Center the running variable at the cutoff, keep a window (bandwidth) around it, and let each side have its own line:

```
bw <- 5
local <- accounts |>
  filter(abs(arr_prior_k - 20) <= bw) |>
  mutate(x_c = arr_prior_k - 20)

rd_model <- lm(renewal_spend_pct ~ x_c * csm, data = local)
tidy(rd_model, conf.int = TRUE)
```

```
# A tibble: 4 x 7
  term        estimate std.error statistic    p.value conf.low conf.high
  <chr>          <dbl>     <dbl>     <dbl>      <dbl>    <dbl>     <dbl>
1 (Intercept)   94.0       0.882    107.   0           92.3       95.8
2 x_c            0.565     0.299      1.89 0.0591      -0.0217     1.15
3 csm            6.02      1.24       4.84 0.00000173   3.58       8.47
4 x_c:csm       -0.444     0.429     -1.03 0.301       -1.29       0.399
```

The `csm` coefficient is the vertical gap between the two fitted lines at the cutoff: about +6.0 points, 95% interval roughly $[3.6, 8.5]$. Not +10.3 — the naive comparison was crediting the CSM program with account size.

This is the local curve-fitting machinery from the smoothing module with one addition: the curve is allowed to break at the cutoff, and the size of the break is the estimate. The bandwidth is the same tradeoff as always. A narrow window compares cleaner neighbors but holds fewer accounts; a wide window adds data but asks a straight line to survive curvature.

```
rd_jump <- function(bw) {
  local <- accounts |>
    filter(abs(arr_prior_k - 20) <= bw) |>
    mutate(x_c = arr_prior_k - 20)
  lm(renewal_spend_pct ~ x_c * csm, data = local) |>
    tidy(conf.int = TRUE) |>
    filter(term == "csm") |>
    mutate(bandwidth = bw)
}

map_dfr(c(2, 3, 4, 5, 6, 8, 10), rd_jump) |>
  select(bandwidth, estimate, conf.low, conf.high)
```

```
# A tibble: 7 x 4
  bandwidth estimate conf.low conf.high
      <dbl>    <dbl>    <dbl>     <dbl>
1         2     3.58   -0.244      7.40
2         3     5.14    1.92       8.37
3         4     6.18    3.45       8.92
4         5     6.02    3.58       8.47
5         6     5.74    3.49       7.99
```

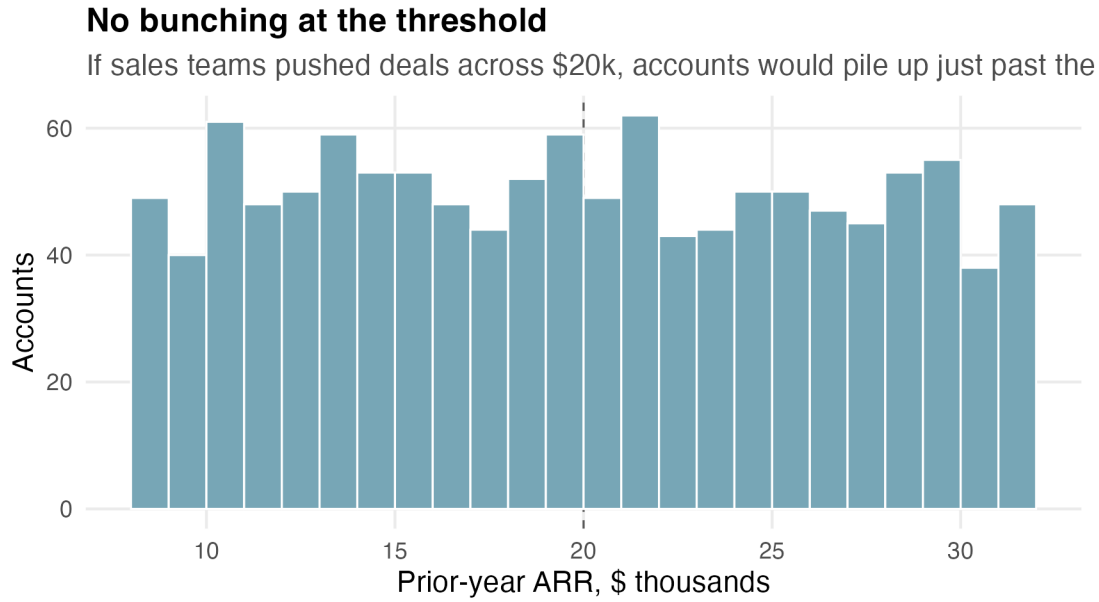
```
6          8     5.91    3.96        7.86
7         10     6.10    4.38        7.81
```

At a $2k bandwidth the estimate is honest but noisy; from $3k on it settles near +6. A global quadratic with a jump gives about +5.9. Report the sensitivity table, not one privileged number.

RD assumptions and the bunching check

The design assumes everything else about an account moves smoothly through $20k. Two failure modes matter in practice. First, other perks that kick in at the same line — priority support, better SLAs — would ride along inside the jump. Second, and sneakier, accounts may *sort* across the line: if sales reps pad contracts to push clients past $20k, then accounts just above the cutoff are the ones with pushy reps, not fair comparisons.

Sorting leaves a signature: a pile-up of accounts just past the line.



No bunching here. A spike just above the cutoff would have been disqualifying.

One more honesty requirement: the jump estimates the CSM effect *for accounts near $20k*. It says nothing direct about a $500k enterprise account. Often the local answer is the decision-relevant one anyway — "should we lower the threshold to $15k?" is a question about marginal accounts, exactly the ones RD describes.

Summary

- A naive before/after comparison assumes the world stood still at launch. It rarely did, and in the courier case it produced the wrong sign.
- Difference-in-differences adds a comparison group that absorbs shared shocks. The regression form is a group-by-post interaction, and the interaction coefficient is the change in changes.

- Parallel trends — the treated group would have changed like the comparison group — is the price of admission. Check its pre-period fingerprints, and investigate co-events, composition changes, anticipation, and targeted timing.
- Regression discontinuity exploits a threshold rule: units just across the line are fair comparisons for units just below it. The treatment effect is the jump at the cutoff, estimated by local lines on each side.
- Bandwidth choice is a bias-variance tradeoff; report estimates across bandwidths. Check that nothing else changes at the line and that the running variable shows no bunching.
- Both designs deliver local estimates — DiD for this rollout in this market, RD for units near the cutoff. Say so in the write-up.